Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Платформенные решения для интеллектуального анализа больших данных»

КАФЕДРА «Компьютерные системы и сети»

ОТЧЕТ

по домашней работе № 1

по курсу «Математические методы анализа данных и принятия решений»

на тему: «Бинарная кластеризация классическим алгоритмом k-means»

Вариант № 8

Студент ИУ6-13М _____ Соболев Н. А.
(Группа) (Подпись, дата) (И. О. Фамилия)

Преподаватель _____ Тверская Е. С.
(Подпись, дата) (И. О. Фамилия)

2026 г.

1 Цель работы

классическим алгоритмом k-means провести процесс бинарной кластеризации предложенных множеств. Проанализировать полученный результат, сделать выводы.

2 Постановка задачи и исходные данные

В соответствии с Вариантом № 8 исходная выборка представляет собой двухмерный массив объектов объема $N = 100$ точек, состоящий из двух кластеров ($K = 2$) со следующими генеральными параметрами:

– Истинные центры кластеров: $\mu_1 = (0.20, 3.00)$, $\mu_2 = (1.00, 3.00)$;
– Среднеквадратические отклонения первого кластера: $\sigma_1 = (0.07, 0.50)$;
– Среднеквадратические отклонения второго кластера: $\sigma_2 = (0.06, 0.50)$;
– Начальные опорные объекты: $z_1^{(0)} = X[1]$, $z_2^{(0)} = X[3]$.

Кластеры анизотропны и сильно вытянуты вдоль вертикальной оси x_2 ($\sigma_y \gg \sigma_x$), при этом расстояние между центрами по горизонтальной оси составляет $\Delta x_1 = 0.8$.

3 Теоретический базис алгоритма k-means

Алгоритм k-means осуществляет разбиение множества объектов на K кластеров $\{S_1, \ldots, S_K\}$ путем минимизации суммарного квадратичного отклонения:

$$J = \sum_{j=1}^{K} \sum_{x \in S_j} ||x - m_j||^2 \to \min \tag{1}$$

где m_j — центроид (вектор выборочного среднего) кластера S_j.

В классической реализации используется стандартное евклидово расстояние:

$$d_E(x, z) = \sqrt{\sum_{k=1}^{n} (x^{(k)} - z^{(k)})^2} \tag{2}$$

Применение евклидова расстояния теоретически обосновано только при условии сферичности порождающих распределений, когда ковариационная матрица имеет вид $\Sigma = \sigma^2 I$.

Для количественной оценки качества разбиения используется внутрикластерная сумма квадратов (WCSS — Within-Cluster Sum of Squares):

$$\text{WCSS} = \sum_{j=1}^{K} \sum_{x \in S_j} d(x, z_j)^2 \tag{3}$$

4 Эксперимент 1. Базовая реализация алгоритма k-means

Программная реализация классического алгоритма k-means на языке Python представлена ниже:

```
import numpy as np
from sklearn.datasets import make_blobs

# Генерация данных Вариант( 8)
np.random.seed(0)
X, _ = make_blobs(
n_samples=100,
centers=[(0.2, 3), (1.0, 3)],
cluster_std=[(0.07, 0.5), (0.06, 0.5)],
random_state=3
)

# Инициализация центроидов
z1 = np.array([X[1], X[3]]).copy()

for iteration in range(100):
dist0 = np.linalg.norm(X - z1[0], axis=1)
dist1 = np.linalg.norm(X - z1[1], axis=1)
labels1 = np.where(dist0 < dist1, 0, 1)

new_z = np.array([X[labels1 == 0].mean(axis=0),
X[labels1 == 1].mean(axis=0)])

if np.allclose(z1, new_z, atol=1e-5):
break
z1 = new_z.copy()

# Расчет ошибки WCSS для классического k-means
wcss_classic = np.sum([np.sum((X[labels1 == i] - z1[i])**2) for i
— in range(2)])
```

Результат работы классического алгоритма представлен на рисунке 1. Вычисленные центры сошлись к значениям $z_1 \approx (0.43, 2.58)$ и $z_2 \approx$

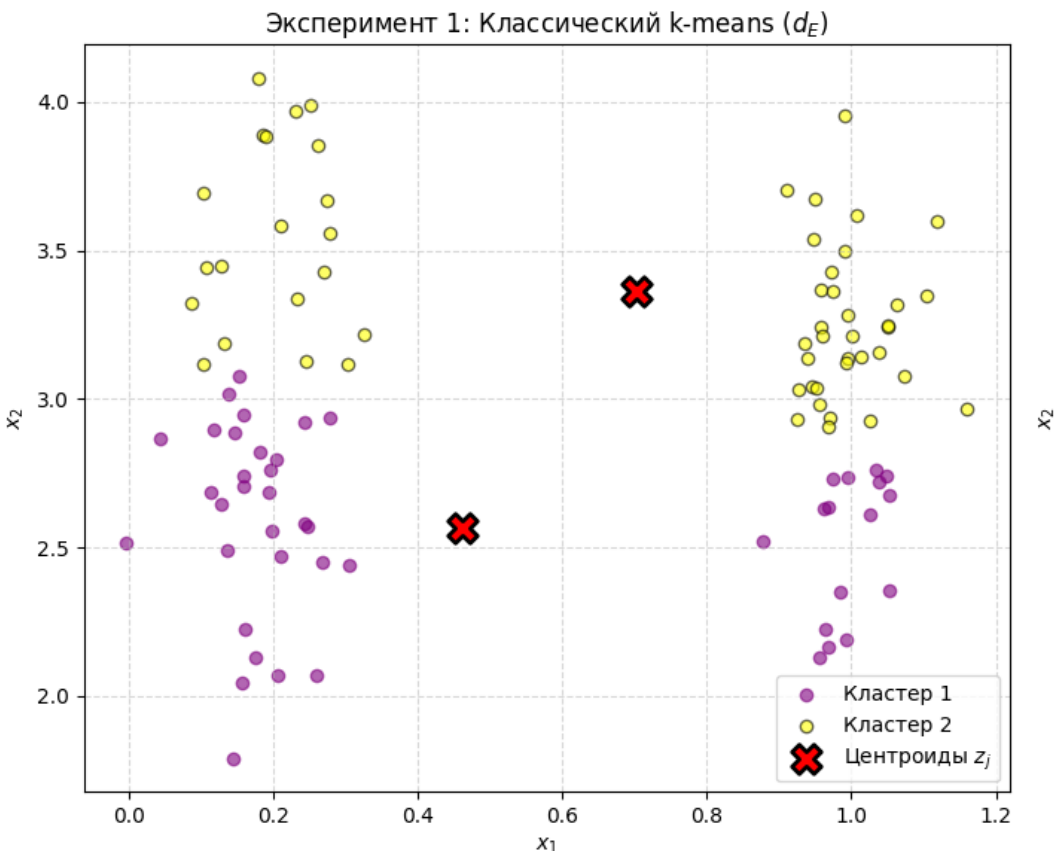


Рис. 1 – Результат работы классического алгоритма k-means (d_E)

$(0.72, 3.36)$. Разделяющая граница прошла по диагонали, разрезав оба естественных кластера.

5 Эксперимент 2. Модификация метрики расстояния

Для устранения влияния высокого разброса по оси ординат x_2 использовано взвешенное евклидово расстояние:

$$d_{wE}(x, z) = \sqrt{\sum_{k=1}^{n} w_k (x^{(k)} - z^{(k)})^2} \tag{4}$$

где вектор весовых коэффициентов принят равным $w = (1.0, 0.1)$.

Программный код взвешенного алгоритма:

```
        w = np.array([1.0, 0.1])
        z2 = np.array([X[1], X[3]]).copy()

        for iteration in range(100):
        dist0_w = np.sqrt(np.sum(w * ((X - z2[0])**2), axis=1))
        dist1_w = np.sqrt(np.sum(w * ((X - z2[1])**2), axis=1))
        labels2 = np.where(dist0_w < dist1_w, 0, 1)

        new_z = np.array([X[labels2 == 0].mean(axis=0),
        X[labels2 == 1].mean(axis=0)])

        if np.allclose(z2, new_z, atol=1e-5):
        break
        z2 = new_z.copy()

        # Расчет ошибки WCSS для взвешенного k-means
        wcss_weighted = np.sum([np.sum(w * (X[labels2 == i] - z2[i])**2)
−   for i in range(2)])
```

Результат кластеризации с использованием метрики d_{wE} приведен на рисунке 2.

Введение весовых коэффициентов позволило провести разделяющую границу строго вертикально. Найденные центроиды $z_1 \approx (0.19, 2.98)$ и $z_2 \approx (1.00, 3.01)$ совпали с истинными центрами генерации.

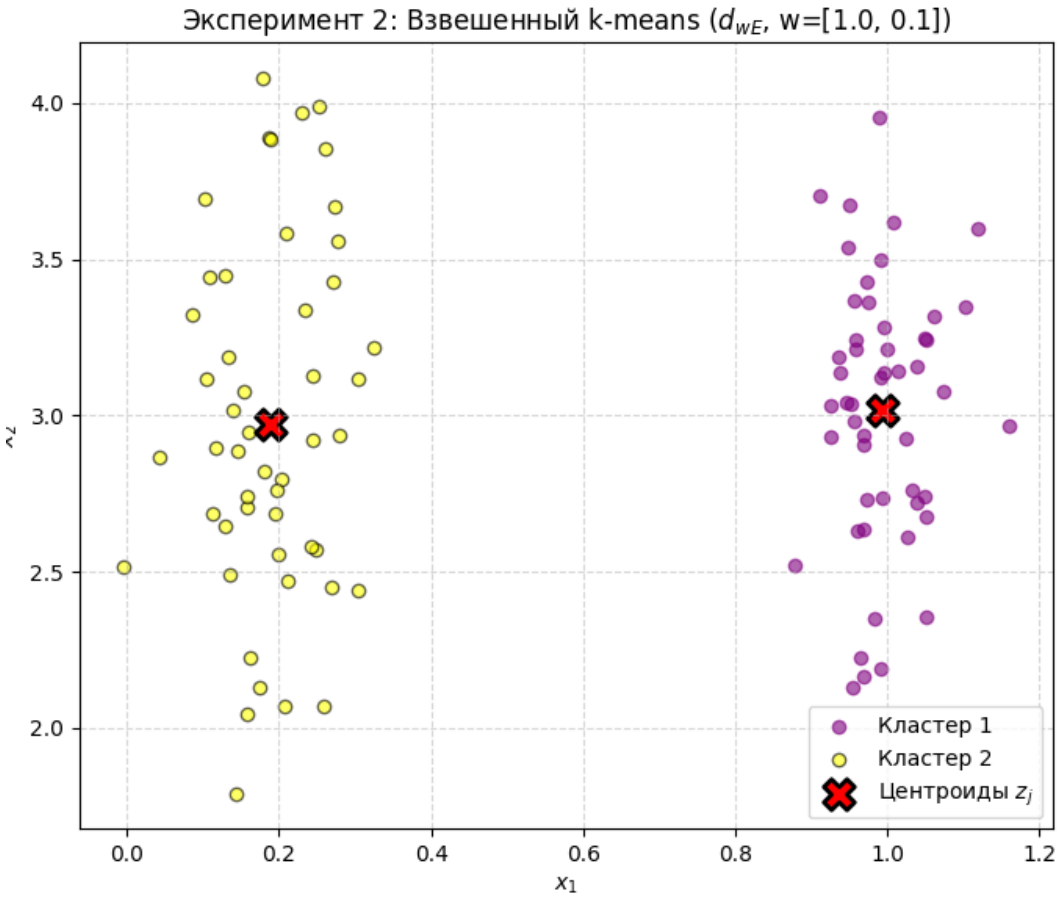


Рис. 2 – Результат работы алгоритма k-means с метрикой d_{wE}

6 Сравнительный анализ и оценка качества

Для количественного обоснования эффективности предложенного подхода произведено сравнение суммарных внутрикластерных ошибок WCSS. На рисунке 3 показано сравнение величины WCSS для обоих методов.

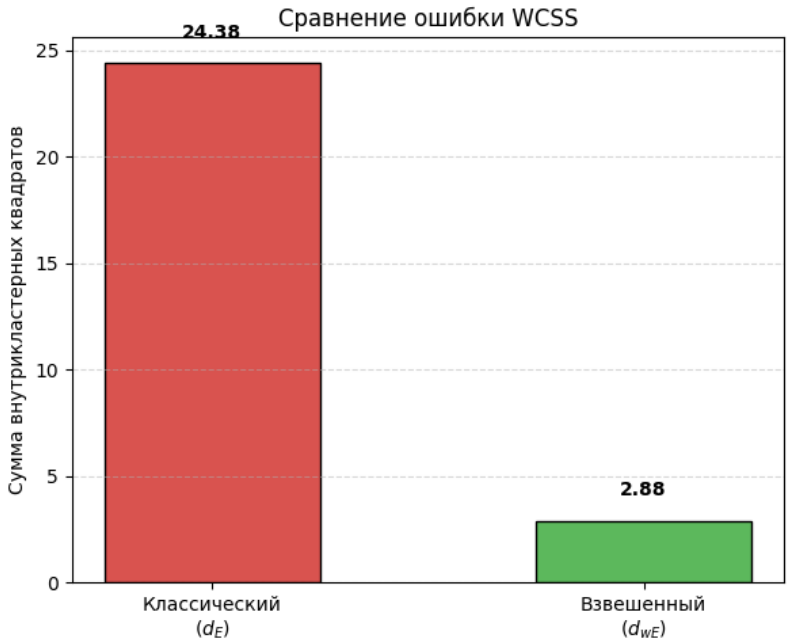


Рис. 3 – Сравнение ошибки WCSS для классического и взвешенного k-means

Сводные результаты проведенных экспериментов приведены в таблице 1.

Таблица 1 – Сравнение точности и метрик алгоритмов

Параметр	**Истинный**	**Классический** d_E	**Взвешенный** d_{wE}
Центроид z_1	$(0.20, 3.00)$	$(0.43, 2.58)$	$(0.19, 2.98)$
Центроид z_2	$(1.00, 3.00)$	$(0.72, 3.36)$	$(1.00, 3.01)$
Граница раздела	Вертикальная	Диагональная	Вертикальная
Ошибка WCSS	—	24.38	2.88

7 Вывод

В ходе выполнения работы были сделаны следующие заключения:

1) **Причина неэффективности классического k-means:** Использование евклидовой метрики d_E предполагает сферическую симметрию кластеров. На анизотропных данных ($\sigma_y/\sigma_x \approx 8.33$) алгоритм разрезает вертикальные кластеры по диагонали, что приводит к некорректному определению центроидов и высокой внутрикластерной ошибке (WCSS $= 24.38$).
2) **Эффективность весовой коррекции:** Введение весов $w = (1.0, 0.1)$ компенсирует разницу масштабов по осям, снижая внутрикластерную ошибку почти в 8.5 раз (WCSS $= 2.88$) и обеспечивая полное восстановление истинного положения центроидов $z_1 \approx (0.19, 2.98)$ и $z_2 \approx (1.00, 3.01)$.